

Technical Engine Specification: PLOS Backend

PLOS Core Architecture Team

September 2026

1 System Architecture Overview

The PLOS backend is engineered following the Clean/Hexagonal Architecture pattern, enforcing a strict separation of concerns across software layers. The entire system is compiled into a single, static CGO-free binary targeting **Go 1.25.0**.

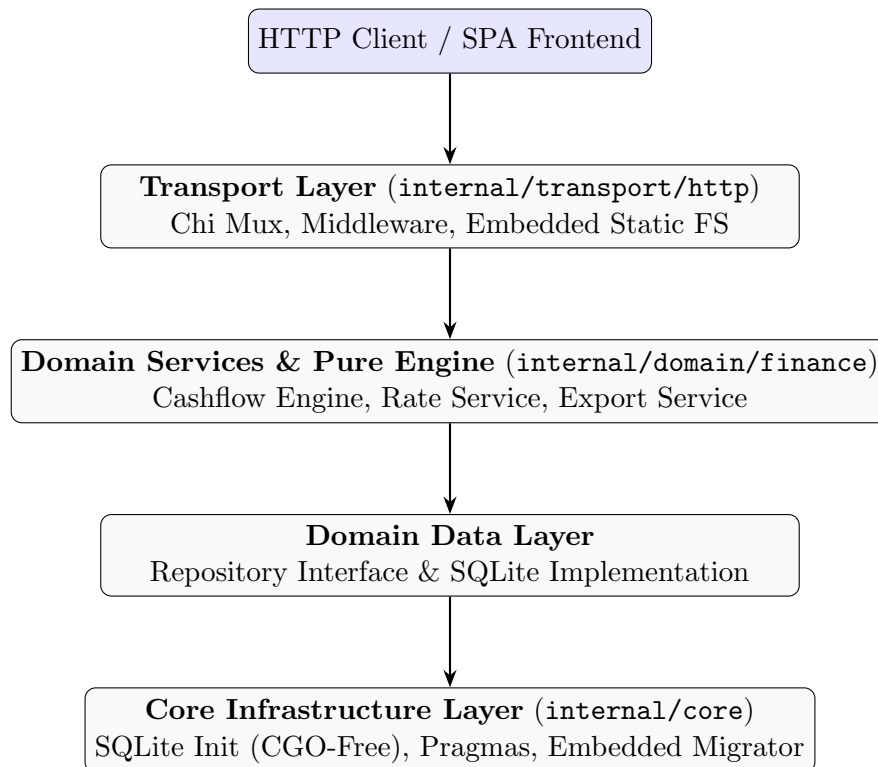


Figure 1: PLOS Backend Hexagonal Architecture Execution Pipeline.

1.1 Package Domain Responsibilities

Package	Domain Area	Mechanical Purpose
cmd/server	Application Entrypoint	Initializes logger (<code>slog</code>), configuration, database pools, migrator, and HTTP server with graceful shutdown handling.
internal/config	Configuration	Loads runtime environment variables (<code>DB.PATH</code>).

Package	Domain Area	Mechanical Purpose
<code>internal/core</code>	Storage Infrastructure	Manages CGO-free SQLite initialization, SQLite Pragma, and transactional migration execution.
<code>internal/domain/finance</code>	Domain Models & Logic	Defines domain entities, Repository contracts, SQLite repository, Rate service, and deterministic Cashflow Engine.
<code>internal/transport/http</code>	Transport / REST API	REST endpoints via <code>chi.Mux</code> , JSON serialization, request validation, and error handling.
<code>migrations</code>	Database Schemas	Versioned SQL migration files embedded directly into the binary using <code>embed.FS</code> .
<code>web</code>	Static Assets Integration	Holds embedded filesystem (<code>embed.FS</code>) of compiled frontend assets (<code>web/dist</code>).

Table 1: Package responsibilities and layer bounds.

2 System Level & Storage Subsystem Mechanics

2.1 CGO-Free SQLite Driver & Database Connection Pool

The system utilizes `modernc.org/sqlite`, a pure Go SQLite implementation created via C-to-Go transpilation. This eliminates runtime dependencies on CGO, enabling true cross-platform `CGO_ENABLED=0` static compilation and removing context-switching overhead between the Go goroutine stack and the C execution stack.

The connection pool is intentionally restricted to prevent file-level lock contention:

$$\text{MaxOpenConns} = 1, \quad \text{MaxIdleConns} = 1, \quad \text{ConnMaxIdleTime} = 5 \text{ minutes} \quad (1)$$

Restricting SQLite writes to a single open connection serializes write operations at the application level, completely preventing `SQLITE_BUSY` lock errors.

2.2 SQLite Hardware Sympathy Configuration (Pragmas)

The database is initialized using optimal SQLite Pragma, tuned for SSD/NVMe storage mechanics:

- `journal_mode(WAL)`: Enables Write-Ahead Logging. Readers do not block writers, and writers do not block readers, achieving concurrent execution throughput.
- `synchronous(NORMAL)`: Eliminates unnecessary `fsync()` system calls on every transaction commit, syncing only during WAL checkpointing.
- `busy_timeout(5000)`: Enforces an internal 5000ms retry backoff loop when lock contention occurs.
- `foreign_keys(1)`: Enforces relational foreign key constraints on every mutation.

3 Cashflow Engine Mechanics

The `CashflowEngine` is a pure, deterministic domain engine free from I/O dependencies or system calls.

3.1 Mathematical Model of the Two-Phase Cascade Algorithm

Let the exchange rate for EUR/UAH be R_{buy} . Foreign currency amounts are normalized to the base currency (EUR):

$$\text{Amount}_{\text{EUR}} = \begin{cases} \text{Amount}, & \text{if Currency} = \text{EUR} \\ \frac{\text{Amount}}{R_{\text{buy}}}, & \text{if Currency} = \text{UAH} \end{cases} \quad (2)$$

The total available liquidity for the monthly period is calculated as:

$$C_{\text{avail}} = \sum \text{Income}_{\text{EUR}} - \sum \text{Expense}_{\text{EUR}} \quad (3)$$

3.1.1 Phase A: Mandatory Minimum Payments

For each active debt $d_i \in D$, the mandatory minimum payment is applied regardless of priority:

$$P_{\min,i} = \min(\text{MinPayment}_{\text{EUR}}(d_i), \text{Remaining}_{\text{EUR}}(d_i), C_{\text{avail}}) \quad (4)$$

$$\text{Remaining}_{\text{EUR}}(d_i) \leftarrow \text{Remaining}_{\text{EUR}}(d_i) - P_{\min,i} \quad (5)$$

$$C_{\text{avail}} \leftarrow C_{\text{avail}} - P_{\min,i} \quad (6)$$

3.1.2 Phase B: Cascade Priority Surplus Distribution

Iterating through active debts sorted in strict priority order ($P_1 \rightarrow P_n$), remaining available liquidity C_{avail} is cascaded:

$$P_{\text{extra},k} = \min(C_{\text{avail}}, \text{Remaining}_{\text{EUR}}(d_k)) \quad (7)$$

$$\text{Remaining}_{\text{EUR}}(d_k) \leftarrow \text{Remaining}_{\text{EUR}}(d_k) - P_{\text{extra},k} \quad (8)$$

$$C_{\text{avail}} \leftarrow C_{\text{avail}} - P_{\text{extra},k} \quad (9)$$

To avoid floating-point representation anomalies (ϵ -precision errors):

$$\forall d_i \in D, \quad \text{if } \text{Remaining}_{\text{EUR}}(d_i) \leq 0.01 \implies \text{Remaining}(d_i) = 0, \quad \text{Status}(d_i) = \text{PAID} \quad (10)$$

4 REST API Specification

HTTP Method	Endpoint Path	Status Code	Description / Purpose
GET	/api/v1/currencies	200 OK	Fetch supported ISO currencies.
POST	/api/v1/currencies	201 Created	Register a new ISO currency reference.
GET	/api/v1/rates	200 OK	Query latest exchange rates.
POST	/api/v1/rates/sync	200 OK	Trigger rate sync via Monobank Public API.
POST	/api/v1/rates/manual	200 OK	Set manual rate override.
GET	/api/v1/accounts	200 OK	Retrieve financial accounts list.
POST	/api/v1/accounts	201 Created	Create new financial account/category.
GET	/api/v1/debts	200 OK	Get active debts ordered by priority ($P_1 \rightarrow P_n$).
POST	/api/v1/debts	200 OK	Upsert debt metadata attributes.

HTTP Method	Endpoint Path	Status Code	Description / Purpose
POST	/api/v1/cashflow/simulate	200 OK	Run one-step cascade payoff simulation.
GET	/api/v1/export/debts.csv	200 OK	Export active debts as RFC 4180 CSV stream.

Table 2: API Route Index and Endpoint Definitions.